Семинар 5. OpenMP: критические секции

1 Понятие о критической секции

Критическая секция — это участок кода программы, который может вызвать состояние гонки (race condition), если к нему одновременно имеют доступ несколько потоков. Для предотвращения этой проблемы используются механизмы синхронизации, которые в `OpenMP` представлены следующими языковыми конструкциями:

1. Директива `critical`.
2. Директива `atomic`.
3. Тип данных «замок» (lock), он же мьютекс.

2 Директива `critical`

Директива `critical` гарантирует, что следующий за ней структурный блок может выполняться только одним потоком одновременно. Так, в программе из листинга 1 строки 3 – 5 будут выполняться первым потоком, который дойдет до этого блока программы, пусть это будет поток 3, а все остальные потоки «зависнут» в строке 1, ожидая, пока третий поток завершит выполнять критическую секцию. После того, как третий поток

Листинг 1: Критическая секция

```
1 #pragma omp critical
2 {
3         f();
4         g();
```

```
5           t();
6   }
```

Если в программе несколько директив `critical`, то все блоки под этими директивами объединяются в единую критическую секцию. Если это не то поведение, которое требуется по условию задачи, то нужно использовать именованную директиву `critical`. Имя указывается в скобках после директивы, т.е. `#pragma omp critical(m)`. В этом случае объединяться будут только критические секции, имеющие одно имя. Так, при выполнении программы в листинге 2 нулевой поток может выполнять инструкции в строках 3 – 5 одновременно с тем, как первый поток выполняет 8 строчку. Однако два потока не могут одновременно выполнять строки 3 – 5 и 11 – 12.

Листинг 2: Именованные критические секции

```
1  #pragma omp critical(one)
2  {
3           f();
4           g();
5           l();
6  }
7  #pragma omp critical(two)
8           k();
9  #pragma omp critical(one)
10 {
11          x();
12          z();
13 }
```

Отметим, что использовать вложенные команды `critical` нельзя. Явное вложение одного блока `critical` в другой прямо запрещено. Программа в листинге 3 компилятором `gcc` попросту не скомпилируется.

Листинг 3: Вложенные одноименные критические секции (ошибка компиляции)

```
int main(){

#pragma omp critical
{
```

```
#pragma omp critical
{
}
}

}
```

Чуть более завуалированное вложение в листинге 4 пройдет проверку компилятора, но зато ожидаемо приведет к взаимной блокировке потоками друг друга. Поясним причину взаимной блокировки. Пусть поток 3 первым попал в строку 11. Все остальные потоки остаются ждать в строке 9, пока третий не закончит выполнять критическую секцию, которая состоит в данном случае из строк 10 – 12 и 3 – 4. Далее, ни один поток не уйдет дальше 2 строки, пока третий не закончит выполнение критической секции. Но это же относится и к третьему потоку. Следовательно, даже третий поток никогда не уйдет дальше второй строки. Данный силлогизм и обосновывает дедлок.

Листинг 4: Вложенные одноименные критические секции (дедлок)

```
1  void f(){
2  #pragma omp critical
3  {
4  }
5  }
6
7  int main(){
8
9  #pragma omp critical
10 {
11         f();
12 }
13
14 }
```

3 Директива `atomic`

Директива `atomic` используется, если критическая секция состоит из одной команды вида `var <op>=<expr>`, либо пост- или преинкремент

или декремент. Эта директива может быть намного более быстрой, чем `critical`.

Различные блоки `atomic` могут как объединяться одну критическую секцию, так и не объединяться в зависимости от реализации. Однако, если в нескольких директивах происходит обновление одной и той же переменной, то объединение блоков в один происходит обязательно в любой реализации. Это показано в листинге 5. В этой программе все три строки (2, 4 и 6) могут быть объединены в одну критическую секцию и выполняться последовательно. Впрочем, этого может и не произойти. Но даже тогда строки 2 и 4 будут выполняться только одним потоком в один момент времени, то есть если один поток выполняет строку 2, то никакой другой не будет выполнять ни 2, ни 4. Однако в это же время любой другой поток может выполнять строку 6.

Листинг 5: Несколько блоков `atomic`

```
1 #pragma omp atomic
2 x += 1;
3 #pragma omp atomic
4 x += 2;
5 #pragma omp atomic
6 y += 1;
```

Отметим, что блок `atomic` не объединяется с блоками `critical` в единую критическую секцию, стало быть, эти блоки могут выполняться одновременно несколькими потоками. Так, в листинге 6 строки 2 и 4 могут выполняться одновременно двумя или более потоками. Следовательно, в этих строках возникает состояние гонки.

Листинг 6: Демонстрация одновременного использования `critical` и `atomic`

```
1 #pragma omp atomic
2 x += 1;
3 #pragma omp critical
4 x = 2;
```

4 Замки (мьютексы, локи)

Мьютекс, он же лок (lock), или замок — это объект типа `omp_lock_t`, над которым можно выполнить две операции: `set` (получение мьютекса) и `unset` (освобождение мьютекса). Когда какой-либо поток получает мьютекс, то он им владеет до того момента, пока сам не освободит его. Любой другой поток, вызвав функцию `set`, не сможет ее покинуть, пока тот поток, который ее первым вызвал, не вызовет `unset`.

Обратите внимание, что если другой поток, отличный от того, который вызвал `set` первым, попытается освободить мьютекс, вызвав `unset` для него, то эта попытка не увенчается успехом. Концепция мьютекса включает в себя идею обладания ресурсами. Поток, впервые сделавший операцию `set` над мьютексом, получает полную над ним власть, и только он может этот мьютекс вновь сделать доступным для других потоков. Эту идею иллюстрирует программа в листинге 7, программа в котором приведет к блокировке первого или второго потока.

Листинг 7: Другой поток не может освободить мьютекс

```
omp_lock_t lock;
omp_init_lock(&lock);

#pragma omp parallel num_threads(3)
{
        int thread_num = omp_get_thread_num();

        if(thread_num == 1 || thread_num == 2){
                omp_set_lock(&lock);
        }
        else{
                omp_unset_lock(&lock);
        }

printf("%d\n", thread_num);
```

В целом работа с мьютексом (замком) происходит по схеме, представленной в листинге 4: создается объект типа `omp_lock_t`, затем для него выделяются ресурсы, которые после работы с мьютексом освобождаются. Далее критическая секция обрамляется вызовами функций `omp_set_lock` и `omp_set_unlock`.

```
omp_lock_t lock;
omp_lock_init(&lock);
omp_set_lock(&lock);
critical section
omp_set_unlock(&lock);
omp_lock_destroy(&lock);
```

5 Пример: пересылка сообщений

Смоделируем передачу сообщений между потоками. Пусть каждый поток генерирует сообщение и отправляет его случайному адресату. Технически это можно реализовать, создав массив очередей, по одной для каждого потока. Первая попытка написать такую программу показана в листинге 8. Содержимое файла `queue.h`, который содержит объявление очереди и заголовки функций для работы с ней, показано в листинге 9.

Листинг 8: Пересылка сообщений с помощью очереди (первая попытка)

```
1  #include <omp.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "queue.h"
5
6  void Send_msg(struct queue_s* msg_queues[], int my_rank,
7      int thread_count){
8    int mes = -my_rank;
9    int dest = rand() % thread_count;
10 #pragma omp critical
11   Enqueue(msg_queues[dest], my_rank, mes);
12 }
13
14 void Try_recv(struct queue_s* q_p, int my_rank){
15   int src, mes;
16   int queue_size = q_p->enqueued - q_p->dequeued;
17   if(queue_size == 0)
18     return;
19   else if(queue_size == 1)
20 #pragma omp critical
```

```
21      Dequeue(q_p, &src, &mes);
22    else
23      Dequeue(q_p, &src, &mes);
24    printf("Thread %d > received %d from %d\n", my_rank, mes, src);
25  }
26
27  int Done(struct queue_s* q_p, int done_sending, int thread_count){
28    int queue_size = q_p->enqueued - q_p->dequeued;
29    if(queue_size == 0 && done_sending == thread_count)
30      return 1;
31    else
32      return 0;
33  }
34
35  int main(int argc, char** argv){
36
37    int thread_count = strtol(argv[1], NULL, 10);
38    int send_max = strtol(argv[2], NULL, 10);
39    struct queue_s** msg_queues =
40      malloc(thread_count * sizeof(struct queue_s));
41
42    int done_sending = 0;
43
44    #pragma omp parallel num_threads(thread_count)
45    {
46      int my_rank = omp_get_thread_num();
47      msg_queues[my_rank] = Allocate_queue();
48
49      #pragma omp barrier
50
51      for(int i = 0; i < send_max; ++i){
52        Send_msg(msg_queues, my_rank, thread_count);
53        Try_recv(msg_queues[my_rank], my_rank);
54      }
55  #pragma omp atomic
56      done_sending++;
57      while(!Done(msg_queues[my_rank], done_sending, thread_count))
58        Try_recv(msg_queues[my_rank], my_rank);
```

```
59      Free_queue(msg_queues[my_rank]);
60      free(msg_queues[my_rank]);
61    }
62    free(msg_queues);
63
64  }
```

Листинг 9: Содержимое файла `queue.h`

```
#ifndef _QUEUE_H_
#define _QUEUE_H_

struct queue_node_s {
        int src;
        int mesg;
        struct queue_node_s* next_p;
};

struct queue_s{
        int enqueued;
        int dequeued;
        struct queue_node_s* front_p;
        struct queue_node_s* tail_p;
};

struct queue_s* Allocate_queue(void);
void Free_queue(struct queue_s* q_p);
void Print_queue(struct queue_s* q_p);
void Enqueue(struct queue_s* q_p, int src, int mesg);
int Dequeue(struct queue_s* q_p, int* src_p, int* mesg_p);
int Search(struct queue_s* q_p, int mesg, int* src_p);

#endif
```

Комментарии к коду.

1. Общая логика работы программы видна из функции `main`. Из командной строки считываем количество потоков и количество сообщений, которые отправляет каждый поток. Каждый поток формирует сообщение, записывает его в очередь своему адресату, опреде-

ляемому случайным образом, а затем проверяет, есть в его очереди не прочитанные сообщения. Если есть, то убирает их из начала очереди и выводит на экран. Эти два действия (отправка сообщения и проверка и вывод) чередуются друг с другом `send_max` раз. После окончания цикла радоваться рано, ибо еще нужно проверить, а не осталось ли непрочитанных сообщений (если какие-то попытки прочитать их в цикле были безуспешными). Для этого вновь вызываем функцию `Try_recv` до тех пор, пока сообщения не кончились.

2. Функция `Send_msg` выбирает случайного адресата и вставляет в конец его очереди сообщение. Т.к. несколько потоков могут выбрать одного и того же адресата и попытаться одновременно поставить сообщение в конец его очереди, то возникает необходимость обеспечить взаимное исключение при выполнении строки 11, что и достигается командой `critical`.

3. Функция `Try_recv` проверяет, есть ли сообщение, и если да, то вынимает его из начала очереди и выводит на экран. Отметим, что операция удаления элемента из очереди выполняется только тем потоком, которому эта очередь принадлежит. Поэтому эту операцию нет нужды заключать в блок `critical`, за исключением частного случая, когда размер очереди равен 1. Тогда одновременное выполнение операции постановки в очередь и удаления из очереди, выполненные разными потоками, приведет к одновременной модификации указателей на начало и конец очереди, которые, в данном случае, совпадают, то есть возникнет состояние гонки. Обратите внимание на то, что здесь мы существенно опираемся на тот факт, что код в строке 21 и 11 составляет единую критическую секцию и одновременное его выполнение невозможно.
   Отметим, что представление размера очереди в 16 строке как разности вставок и удалений позволяет одновременно без негативных последствий делать `Dequeue` и `Enqueue`, т.к. эти операции изменяют разные счетчики. Иначе, если мы хранили в очереди переменную `size` в готовом виде, одновременная вставка и удаление привели бы к гонке потоков за размер очереди.

4. Функция `Done` проверяет, все ли запланированные сообщения были отправлены. Обратите внимание, что второй операнд конъюнкции

в строке 29 критически важен, ибо без него могла бы произойти ситуация, когда какие-то сообщения еще не отправлены (не записаны в очередь), но все записанные уже получены. Что же касается переменной `done_sending`, то она инкрементируется под директивой `atomic`, поэтому ее увеличение не сливается в единую критическую секцию, которую составляют строки 21 и 11.

Проблема программы в листинге 8 заключается в том, что строки 21 и 11 не могут выполняться одновременно. С одной стороны, это как раз то, чего мы добивались, см. комментарий 3. Однако данное ограничение явно избыточно, ибо запрещает делать одновременную вставку и удаление элемента не только в одной очереди, но и в разных! Решим проблему с помощью более «мелкозернистого» примитива синхронизации, а именно, мьютексов. Для этого в объявление очереди добавим мьютекс. Функции работы с очередью оставим почти без изменений, лишь в функцию аллокации добавим вызов `omp_init_lock`, а в функцию освобождения памяти вызов `omp_destroy_lock`. Строки 21 и 11 обрамим получением и освобождением мьютекса, встроенного в очередь. Т.к. для каждой очереди мьютекс свой, то такая конструкция позволит одновременно вставлять и удалять элементы в очереди разных потоков.

6 Выводы по семинару

По возможности следует использовать директиву `atomic`, если критическая секция состоит из одной инструкции вида «измени и запиши».

В противном случае нужно использовать директиву `critical`, в простом или именованном варианте. Нет данных о том, что данная директива медленнее мьютекса, а работать с ней проще.

Наконец, использовать мьютекс в том случае, если требуется более мелкозернистый контроль над работой критической секции.